

Mirage: An Adaptive Cyber Deception and Active Response Platform

Project Proposal

Group Members:

M Amar Waqar	23L-3035
Hussain Nawaz	23L-3018
Haseeb Ahmad	23L-3069

National University Of Computer and Emerging Sciences
Department of Computer Science
Lahore, Pakistan

Abstract

Most network defence today works by watching logs and matching known attack signatures. This creates a flood of alerts, many of them false alarms, and usually spots an intruder long after they have already broken in. Mirage takes the opposite approach. Instead of looking for the attacker in a sea of normal traffic, it sets up fake targets that no real user would ever touch. Mirage is built as one working pipeline. Decoy services in Docker containers (a fake admin panel, a fake internal SSH server and a fake AWS S3 storage endpoint) are joined by uniquely tagged fake secrets into a single breadcrumb trail that leads an intruder deeper into the trap. Every touch sends an event to one controller, which raises a reliable alert, labels it with the matching MITRE ATT&CK technique and MITRE Engage activity, and decides the response. The controller can watch the attacker, slow them down, or shut them out by blocking their address at the firewall and ending their session, while saving the evidence. A live dashboard shows each step along with every automatic response. Mirage will be tested in an isolated lab with a scripted attack, measuring how fast it detects and contains an intruder, how few false alarms it raises, and how many attack techniques it catches.

1. Introduction

Once an attacker gets past the outer defences of a network, they tend to move around quietly, stealing passwords, finding services and copying data out. Reports show that these break-ins often go unnoticed for weeks. The usual defences, such as firewalls, signature based intrusion detection and log analysis tools, are mostly reactive. They have to pick out the few harmful actions from a huge amount of normal activity, which leads to tired analysts and missed attacks.

Cyber deception works differently. It fills the network with decoys (fake systems and services) and honeytokens (fake passwords, keys, files and records) placed where an attacker is likely to look. Because these items have no real purpose, anyone who touches them is almost certainly an intruder, so the alerts are accurate and there are very few false alarms. The MITRE Engage framework describes this way of working as engaging the attacker, with goals such as Expose, Affect and Elicit and methods such as lures and decoy files. Paid products that do this already exist, but they are expensive and closed. Free honeypots exist too, but they are usually single traps that only record what happens and never react.

Mirage sets out to fill this gap with one working pipeline that follows an intruder from the first touch to being shut out. Its main ideas are: (i) a planned breadcrumb trail, so the platform always knows how far an intruder has gone; (ii) fake secrets that are each unique, so every alert shows exactly which item was stolen and from where; and (iii) a range of automatic responses, from quietly watching to fully shutting the attacker out. On top of this, the fake SSH server is powered by a large language model that acts like a real Linux shell, so attackers stay longer and show more of what they are trying to do.

2. Goals and Objectives

The main goal of Mirage is to show that one automatic deception and response pipeline can spot an attacker quickly after a break-in, raise almost no false alarms, and shut the attacker out on its own. The specific objectives are:

- To build and run decoy services in Docker containers: a fake admin web panel, a fake internal SSH server, and a fake AWS S3 storage endpoint.
- To create realistic, uniquely tagged fake secrets (a fake SSH password, fake AWS keys and a bait passwords.xlsx file), plant them along the trail and record where each one was placed.
- To give the fake SSH server an AI driven shell that uses a large language model to produce realistic, consistent command output, so attackers stay longer, and to label every command with its ATT&CK technique.
- To link the decoys and fake secrets into a breadcrumb trail that guides an intruder along a planned path and shows how far they have reached.
- To build a detection engine that receives every decoy event in one common format, raises an instant alert, and labels each event with its MITRE ATT&CK technique and MITRE Engage activity.
- To build a response controller that can watch, slow down, or shut out an attacker, including blocking their address at the firewall for a set time, ending their session, and saving the evidence.
- To build a live dashboard that shows the event feed, progress through the trail, the ATT&CK techniques seen and the automatic responses taken.
- To test the pipeline against a scripted attack and normal traffic, reporting how fast it detects and contains an attacker, its false alarm rate, and how many ATT&CK techniques it catches.

3. Scope of the Project

Mirage will be built and shown entirely inside an isolated Docker based lab network. It is one pipeline, in which a single attack path feeds a single flow of events: decoys, detection, response, dashboard. It has the following parts.

3.1 Decoy Infrastructure

Docker containers running decoy services that are left exposed on purpose. Each one is filled with realistic made up content (staff names, file histories and logs) so it looks like a real, lived in system:

- **Fake admin panel:** a convincing internal login page whose robots.txt points to an exposed backup .env file. Every login attempt and folder scan is recorded.
- **Fake internal SSH server:** a working SSH login with a realistic file system that hides planted secrets (such as ~/.aws/credentials and .bash_history). Any command it does not script directly is handled by the AI driven shell described below, and whole sessions are recorded as evidence.
- **Fake AWS S3 endpoint:** an S3 style storage service built on MinIO that accepts the planted fake AWS keys and holds the bait passwords.xlsx file, so we can watch exactly what an attacker does with stolen cloud keys.

3.2 AI Driven Interactive Shell

Ordinary SSH honeypots only know a fixed list of commands, so skilled attackers spot them quickly and leave. Mirage's fake SSH server mixes two layers:

- **Scripted layer:** common commands and all the planted breadcrumbs (such as ~/.aws/credentials and .bash_history) come from a fixed fake file system, so the bait is always there and always the same.
- **AI layer:** any other command is passed to a large language model that is told to act as one specific fake server (with a set hostname, operating system, services and users) and produces realistic output for commands such as ps, netstat, config files or database clients.
- **Session memory:** anything the attacker changes (new files, folders or downloaded tools) is remembered and fed back to the model, so the answers stay consistent for the whole session.
- **Command labelling:** every command is matched to an ATT&CK technique (such as discovery, credential access, tool download or persistence) and shown live on the dashboard, raising the attacker's risk score.

This keeps the attacker busy for longer and captures more of what they do, which supports the MITRE Engage goal of drawing out attacker behaviour.

3.3 Honeytoken Fabric

A tool that creates realistic fake secrets, gives each one a unique tag and records where it was placed, then spreads them across the environment:

- A fake SSH password in the exposed backup .env file, and fake AWS keys in ~/.aws/credentials, in real looking formats.
- A bait passwords.xlsx file in the fake S3 bucket that raises an alert when it is downloaded.
- A fake link in robots.txt that reveals scanning.

These items are joined into one breadcrumb trail: robots.txt → exposed backup .env file → SSH login → fake SSH server → AWS keys → fake S3 bucket → passwords.xlsx. Because every item is unique, each alert shows exactly which one was stolen, where it came from, and how far along the trail the attacker has reached.

3.4 Detection and Active Response Controller

- **Single event flow:** every decoy sends its events to one controller in one common format.
- **Instant alerts:** every time a decoy or fake secret is touched, Mirage raises an alert on the dashboard that includes the attacker's address, the item touched, and the matching MITRE ATT&CK technique and MITRE Engage activity.
- **Attacker profiling:** each attacker gets a risk score built from how far they have come along the trail and the techniques they have used.
- **Response playbooks:** the risk score picks the response. Watch lets low risk activity continue so Mirage can learn more. Slow down drags the attacker's connection out. Shut out runs the containment sequence below. An allow list makes sure the defenders are never locked out by mistake.

3.5 Containment and Disruption

Once an attacker's risk score crosses the limit, Mirage runs a disruption sequence. Every step targets only the attacker's own session and the decoy containers that Mirage owns, so the whole response stays inside the isolated lab and never reaches any outside machine. The aim is to stop the break-in on the spot, waste the attacker's time, and keep the evidence:

- **Session freeze:** the attacker's connection is slowed to a stop, so their tools stall, hang and time out.

- **Lockout and notice:** the session is ended, a message tells the intruder they have been caught and shut out, and their address is blocked at the firewall using nftables for a set time.
- **Evidence capture:** before the decoy is reset, Mirage saves the full session recording and a snapshot of the container so the break-in can be studied later.

On the dashboard this all happens live. The attacker's progress along the trail stops dead, each disruption step lights up one after another, and the response log fills in real time while the intruder is frozen out.

3.6 Command and Telemetry Dashboard

A single live web page that shows the event feed, how far each attacker has come along the trail, credential theft events, the ATT&CK techniques seen, and a log of the automatic responses.

3.7 Evaluation

The platform will be tested in the lab with an attacker container (such as Kali Linux) running a scripted attack using common tools like nmap, gobuster, hydra and the AWS CLI, alongside a traffic generator that acts like normal users. We will measure the average time to detect and time to contain an attacker, the false alarm rate, and the number of ATT&CK techniques caught.

3.8 Out of Scope

Everything runs inside the isolated lab. Mirage never attacks the intruder's own machine, and it uses only fake cloud accounts and fake passwords, never real ones.

4. Initial Study and Work Done so Far

We have studied the MITRE Engage framework for engaging attackers and the MITRE ATT&CK knowledge base, which we will use to label what attackers do. We read the early work on honeypots and honeytokens, including Spitzner's writing on both and the research on honeyfiles as fake files for catching intruders, along with a survey of deception technology.

We also looked at existing free tools. Cowrie is a widely used fake SSH and Telnet server. Thinkst Canarytokens hands out trackable items such as fake AWS keys and canary documents. T-Pot bundles many honeypots behind one dashboard. Newer projects such as shelLM and Galah show that large language models can make honeypots far more believable. We found that these tools mostly work on their own and only watch: they record what happens but do not join the decoys into a single trail or react on their own. Mirage is built to close these gaps.

From this study we have settled on the overall design and its main parts, planned the breadcrumb trail, and chosen the tools: Docker Compose for the lab, Python with FastAPI and

SQLite for the detection and response controller, a Paramiko based fake SSH server with a language model shell, MinIO for the S3 endpoint, nftables for blocking attackers, and a single web page dashboard that updates live over WebSockets.

References

[1] The MITRE Corporation, “MITRE Engage: A framework for adversary engagement,” 2022. [Online]. Available: https://engage.mitre.org. [Accessed: Oct. 24, 2026].

[2] The MITRE Corporation, “MITRE ATT&CK: Adversarial tactics, techniques, and common knowledge.” [Online]. Available: https://attack.mitre.org. [Accessed: Oct. 24, 2026].

[3] L. Spitzner, Honeypots: Tracking Hackers. Boston, MA: Addison-Wesley, 2002.

[4] L. Spitzner, “Honeytokens: The other honeypot,” SecurityFocus, Jul. 2003. [Online]. Available: https://www.securityfocus.com/infocus/1713.

[5] J. Yuill, M. Zappe, D. Denning, and F. Feer, “Honeyfiles: Deceptive files for intrusion detection,” in Proc. 5th Annu. IEEE SMC Information Assurance Workshop, West Point, NY, 2004, pp. 116–122.

[6] D. Fraunholz et al., “Demystifying deception technology: A survey,” arXiv preprint arXiv:1804.06196, 2018.

[7] M. Sladić, V. Valeros, C. Catania, and S. Garcia, “LLM in the shell: Generative honeypots,” in Proc. IEEE European Symposium on Security and Privacy Workshops (EuroS&PW), 2024.

[8] Cowrie Project, “Cowrie SSH/Telnet honeypot.” [Online]. Available: https://github.com/cowrie/cowrie. [Accessed: Oct. 24, 2026].

[9] Thinkst Applied Research, “Canarytokens.” [Online]. Available: https://github.com/thinkst/canarytokens. [Accessed: Oct. 24, 2026].

[10] Deutsche Telekom Security, “T-Pot: The all-in-one honeypot platform.” [Online]. Available: https://github.com/telekom-security/tpotce. [Accessed: Oct. 24, 2026].

[11] A. Karimi, “Galah: An LLM-powered web honeypot.” [Online]. Available: https://github.com/0x4D31/galah. [Accessed: Oct. 24, 2026].